Wind Energy Forecasting Pipeline Report

Data4Econ Project

30 March 2026

1 Purpose

This report summarizes the wind energy forecasting pipeline implemented in `whole.py`, following the conceptual structure described in `pipeline_design.md`. The goal is to forecast hourly wind power production for the July–December 2025 test horizon using historical production data and multi-provider weather forecasts.

The pipeline has two main ideas:

- not all weather inputs are equally trustworthy at every hour;
- weighted weather features can then be passed to a downstream forecasting model that predicts corrected wind production.

2 Data and Prediction Target

Two input files are used:

- `historical_data.csv`: plant production history, availability, and ODD flags;
- `weather_forecast.csv`: weather forecasts from multiple providers, models, sites, and measures.

The production target used by the code is not raw production. It is the availability-corrected target

$$y_t = \texttt{Production_corr_MWh} = \frac{\texttt{Production_MWh}}{\texttt{Availability}}.$$

Rows with `ODD` different from 1 or `Availability` equal to 0 are removed before training.

The current implementation also collapses duplicated daylight-saving timestamps by averaging duplicate rows at the same datetime. This avoids index inconsistencies during train/validation splitting.

3 Pipeline Overview

Conceptually, the pipeline follows the two-stage design:

$$w(t) = w_{\text{static}} \odot w_{\text{dynamic}}(t),$$

where w_{static} captures long-run feature relevance and $w_{\text{dynamic}}(t)$ captures hour-specific reliability. The resulting weighted weather representation is then used for forecasting.

In implementation terms, the full flow is:

1. load and clean historical and weather data;

2. parse weather columns into comparable and standalone groups;
3. engineer weather features, including unit normalization and wind-vector features;
4. compute a static feature-importance vector from training correlations;
5. build a meta-feature matrix that describes cross-model agreement, anomalies, and temporal context;
6. train a small MLP that produces dynamic feature weights;
7. apply static and dynamic weights to the normalized weather features;
8. append production lags;
9. train a DLinear-style forecasting model based on trend/residual decomposition and Ridge regression;
10. evaluate on a hold-out validation window and generate submission forecasts.

4 Stage 1: Input Reliability Assessment

4.1 Weather schema and pruning

The design document distinguishes between:

- **comparable features**: variables that exist across multiple forecast sources and can therefore be compared against each other;
- **standalone features**: variables such as temperature and irradiation that do not have a symmetric cross-provider comparison structure.

The code parses the raw weather column names into:

- model,
- site,
- raw measure,
- provider,
- duplicate status.

It then keeps:

- comparable measures: wind speed, wind gust, wind direction;
- standalone measures: temperature and irradiation.

Provider3-style sub-variant columns and duplicate suffix columns are dropped. The result in the current run is:

- 118 comparable columns,
- 60 standalone columns,
- 178 retained weather columns in total.

4.2 Feature engineering before weighting

The code performs the main transformations proposed in the design:

- **unit normalization**: wind speed and gust values reported in km/h are converted to m/s;
- **wind direction encoding**: wind speed and direction are combined into u and v vector components;
- **power-curve proxy**: v^3 features are created to reflect the nonlinear relationship between wind speed and power;
- **missing-value handling**: later stages use forward fill, backward fill, and zero fallback for numerical stability.

4.3 Static weights $w_{\mathbf{static}}$

The static weight vector is meant to encode long-run predictive usefulness. In the code, each retained feature receives a score based on:

- absolute Pearson correlation with corrected production;
- absolute Spearman correlation with corrected production;
- the fraction of non-missing observations.

For feature j, the implemented raw score is conceptually:

$$s_j = \left(0.5|\rho_j^{(P)}| + 0.5|\rho_j^{(S)}|\right) \cdot \text{availability}_j.$$

The final static weights are obtained by a global softmax over all retained columns.

This matches the spirit of the design document, although the current implementation is simpler than the full design: it does not explicitly normalize within measure families or aggregate first at set level and site level.

4.4 Dynamic weights $w_{\mathbf{dynamic}}(t)$

The design document proposes a per-hour reliability model driven by disagreement across forecast sets, temporal stability, missingness, anomalies, and time context. The implementation follows that idea through an MLP input matrix composed of:

- cyclic time features: hour-of-day and day-of-year sine/cosine terms;
- cross-set agreement summaries per measure and site: mean, standard deviation, range, number of available sources, and per-column deviation from group mean;
- temporal consistency features for selected v^3 columns: rolling mean, rolling standard deviation, jumps, lags, missing flags, and z-scores;
- optional recent production summaries during training.

The MLP is trained on these meta-features and then converted into dynamic feature weights by projecting its hidden representation to the weather-feature dimension. Those dynamic weights are combined multiplicatively with the static weights and the normalized weather matrix.

This again follows the design conceptually, but the implementation is lighter than the original specification:

- it uses `sklearn`'s `MLPRegressor`;
- it does not implement explicit BatchNorm or Dropout layers;
- it is not trained end-to-end with a dedicated linear-probe objective from the design text.

5 Stage 2: Forecasting Model

The design document leaves Stage 2 open to alternatives such as XGBoost, LSTM/GRU, or hybrids. The implemented code chooses a **DLinear-style forecasting model**:

- the weighted input matrix is decomposed into trend and residual components using a moving average;
- each component is flattened over a rolling look-back window;
- two Ridge regressions are trained, one for trend and one for residual;
- their outputs are added to produce the final normalized forecast.

The current configuration uses:

- look-back window: 48 hours,
- forecast horizon: 24 hours,
- moving-average kernel: 25,
- Ridge regularization parameter: 10^{-2}.

The code also appends lagged production features before Stage 2:

- lag 1 hour,
- lag 24 hours.

To reduce systematic overprediction during low-production periods, the current version adds **weighted Stage 2 training**. Training sequences whose future target window has lower average production receive higher sample weight. This biases the Ridge fit toward better behavior in the low-output regime.

6 Train, Validation, and Test Split

The code uses an internal temporal split:

- **train**: from 1 January 2024 to 30 March 2025,
- **validation**: from 31 March 2025 to 30 June 2025,
- **test**: from 1 July 2025 to 31 December 2025.

Validation is therefore a strict future hold-out relative to the training portion, which is appropriate for time-series forecasting.

7 Evaluation Metrics

The pipeline reports several validation metrics:

- **MAE**: mean absolute error in corrected MWh;
- **RMSE**: root mean squared error in corrected MWh;
- **Bias**: mean signed forecast error in corrected MWh;

- **WAPE_pct**: weighted absolute percentage error,

$$\mathrm{WAPE} = \frac{\sum_t |\hat{y}_t - y_t|}{\sum_t |y_t|} \times 100;$$

- E_f: the hackathon metric,

$$E_f = \sum_D \epsilon_{\mathrm{abs}}(D) + \left| \sum_D \epsilon_{\mathrm{bias}}(D) \right|.$$

Because the target contains many very small or zero-production hours, WAPE can become large and should be interpreted carefully. MAE remains the most direct physical-unit interpretation: it is the average hourly miss measured in corrected MWh.

8 Current Validation Results

For the latest run, the output metrics are:

Metric	Value
MAE	1.6374
RMSE	2.2410
Bias	0.5058
WAPE_pct	114.3300
E_f	4145.0395

These values are computed on the internal validation window only, not on the final hidden competition test set.

9 Recent Fixes and Model Adjustments

Several implementation issues were identified and fixed while running the pipeline:

- **DST duplicate timestamps**: duplicate historical and weather timestamps around daylight-saving fallback were collapsed before splitting;
- **normalized-space clipping bug**: predictions were previously clipped at zero before denormalization, which created an artificial prediction floor near the training mean; clipping now happens only after converting back to MWh;
- **low-production weighting**: Stage 2 now upweights low-output sequences during training to reduce positive bias near zero.

The second fix is especially important conceptually: zero in normalized space is not zero production in MWh. Clipping in normalized space therefore imposed a false lower bound on real predictions.

10 Outputs Produced by the Pipeline

The current implementation writes the following artifacts to the output directory:

- `submission.csv`: hourly test forecasts for July–December 2025;
- `val_metrics.txt`: numerical validation metrics;

- `val_predictions.png`: validation time-series and scatter plot;
- `training_loss.png`: Stage 1 and Stage 2 training curves;
- `pipeline_report.tex`: this report.

11 Relationship to the Design Document

Overall, the code is faithful to the design document at the architectural level:

- Stage 1 estimates feature reliability from both long-run relevance and time-varying trust;
- Stage 2 forecasts production using the reliability-weighted weather representation;
- the weather structure, provider asymmetry, unit normalization, and cross-set disagreement ideas are preserved.

However, the implementation is deliberately more pragmatic than the design document:

- the static weighting is simpler than the set/site normalization proposed in the design;
- the dynamic MLP is a practical surrogate rather than a full custom neural architecture;
- Stage 2 uses DLinear-style Ridge models instead of a neural recurrent model or boosted trees.

This makes the pipeline easier to run, debug, and reproduce while still respecting the central idea of reliability-aware weather fusion for wind power forecasting.

12 Conclusion

Conceptually, the code solves the forecasting problem in two steps: first decide which weather signals deserve trust at each hour, then forecast production from those trust-adjusted signals. The design document provides the reasoning framework, and the current implementation turns that framework into a compact scikit-learn-based system that can be trained and executed end-to-end from the command line.